

Client-Driven Performance Model of Hyperledger Fabric Blockchain via Phase Decomposition

Canhui Wang , *Member, IEEE*, and Xiaowen Chu* , *Fellow, IEEE*

Abstract—Hyperledger Fabric is a popular permissioned blockchain platform. Although many existing works on the overall performance of Hyperledger Fabric are available, a decomposition of each phase in Hyperledger Fabric remains to be explored. While the overall performance of Hyperledger Fabric might provide an end-user with sufficient performance information when processing a transaction, it is far from informative when deploying a distributed system with specific performance goals, except for understanding each decomposition phase in Hyperledger Fabric. This paper presents a client-driven measurement framework to characterize the transactions of each decomposition phase in Hyperledger Fabric, based on the Fabric SDK Node.js. We evaluate the Hyperledger Fabric performance and have some interesting observations, e.g., 1) The number of CPU cores linearly impacts the throughput of an endorsing peer. 2) Raft-based ordering service shows good scalability with the number of ordering service nodes. 3) Waiting time dominates a transaction’s latency in the order phase when encountering unfit block-setting parameters. We then identify the dominant resource in each decomposition phase, propose a stochastic computation model and an alpha-beta communication model to analyze the performance of each decomposition phase. Finally, we validate the performance model’s accuracy in two clusters. And we apply the model and propose a strategy to reduce waiting time when it dominates transaction latency in the order phase under unfit block-setting parameters.

Index Terms—Blockchain, Hyperledger Fabric, Phase Decomposition, Unfit Block-Setting Parameters.

I. INTRODUCTION

BLOCKCHAIN is a distributed ledger technology (DLT) originally proposed to solve the double-spending attack in the Bitcoin (BTC) cryptocurrency [1]–[3]. A blockchain is a chain of blocks replicated among participants on a peer-to-peer (P2P) network. An important property of blockchain is that it does not rely on any third party for transaction clearing; instead, it stores and updates all valid historical transactions for future reference. Thus, it enables determining whether a new incoming transaction is valid. People categorize blockchain into permissioned blockchain [4]–[11] and permissionless blockchain [12]–[19]. The permissioned blockchain requires prior authorization, allowing only authenticated users to access specific blockchain data, whereas the permissionless blockchain does not.

Canhui Wang is with the Department of Computer Science and Technology, Huaqiao University, Xiamen. He is also affiliated with Xiamen Key Laboratory of Data Security and Blockchain Technology, and Key Laboratory of Computer Vision and Machine Learning of Fujian Provincial Universities. E-mail: chwang@hqu.edu.cn

Xiaowen Chu is with the Data Science and Analytics Thrust of The Hong Kong University of Science and Technology (Guangzhou). E-mail: xwchu@hkust-gz.edu.cn

Manuscript received Dec 28, 2025. (*Corresponding author: Xiaowen Chu)

Hyperledger Fabric is a popular permissioned blockchain with multiple components, including peer, ordering service, and smart contract. Each transaction proposed to update the global state of the blockchain database undergoes these components in sequence, except for read-only transactions, which do not update the global state. To reduce component dependency, recent versions of Hyperledger Fabric, i.e., v1.0 and later, employ a loosely coupled architecture known as the execute-order-validate (EOV) model [20]–[23]. It encapsulates these components into pluggable modules and then decouples the overall system into three phases: execute, order, and validate, where the output of one phase becomes the input of the next phase. The execute phase receives new proposals from the client and simulates new transactions for the client. The order phase puts these new transactions from the client into a new block and then forwards the new block to the validate phase. Finally, the validate phase validates each transaction, commits the new block, and replies to the client.

Our motivations for performance modeling and analysis of Hyperledger Fabric blockchain are as follows. First, most existing works studied Hyperledger Fabric blockchains from an overall performance perspective [24]–[29] while few works decompose the three phases in Hyperledger Fabric. Although an overall performance provides an end-user with sufficient information, it is insufficient for a system developer to deploy the Hyperledger Fabric blockchain to meet specific performance goals, except for understanding each decomposition phase. Moreover, existing works lack a measurement framework for phase decomposition [20], [22] in Hyperledger Fabric. Thus, it is important to propose a measurement framework to decompose the three phases in Hyperledger Fabric.

Second, it is important to model each decomposition phase [20], [22]. On one hand, performance modeling of each decomposition phase helps computing resource allocation [30]–[33], such as CPU cores, network bandwidth, and disks, because simply powering non-dominant computing resources cannot improve the throughput of Hyperledger Fabric service. On the other hand, it helps latency analysis [34]–[38] of Hyperledger Fabric service, because simply adding non-parallel dominant computing resources cannot improve the latency of Hyperledger Fabric service; instead, it leads to a waste of computing resources. Thus, it is necessary to identify the dominance and parallelism of computing resources, and to model the performance characteristics of each decomposition phase in Hyperledger Fabric service.

Besides characterizing the performance, e.g., throughput and latency of each decomposition phase, our proposed model helps to reduce waiting time when it dominates transaction

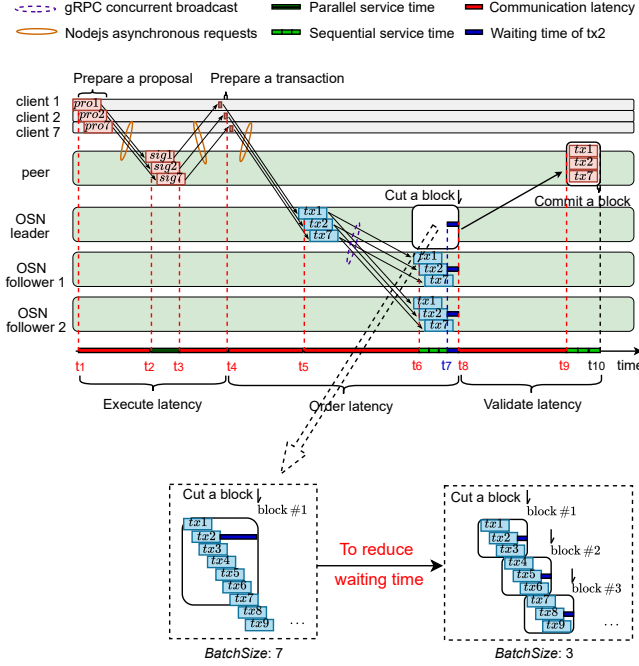


Fig. 1. An illustration of waiting time in our measurement framework.

latency in the order phase under unfit block-setting parameters. A transaction latency typically comprises four parts [39], [40]: propagation latency, transmission latency, queueing latency, and node processing latency. Waiting time is a part of the node processing latency. As illustrated in Fig. 1, the waiting time of a transaction in the order phase is represented in a blue square. For instance, given a deterministic non-parallel service rate of 1 second per transaction, a deterministic transaction arrival interval of 2 seconds, and a batch size of 7 transactions per block. Obviously, a transaction's queueing latency is 0 seconds, and its average waiting time in the order phase is 2 seconds, because transactions must wait until the number of transactions included in a block reaches at least 7. Thus, in the worst case, the waiting time would be non-negligible and even dominate a transaction's latency in the order phase when encountering unfit block-setting parameters. Motivated by this, we aim to reduce waiting time when it dominates transaction latency in the order phase under unfit block-setting parameters. Finally, the main contributions of this paper are summarized as follows,

- We propose a client-driven measurement framework to decompose each phase in the Hyperledger Fabric service, and open-source the framework [41] for Hyperledger Fabric 2.2 LTS.
- We propose a performance model to analyze each decomposition phase in Hyperledger Fabric, and validate its accuracy in two clusters.
- We apply the performance model and propose a strategy to reduce waiting time when it dominates transaction latency in the order phase under unfit block-setting parameters.

The remainder of this paper is organized as follows. Section II reviews related work in performance analysis and modeling of Hyperledger Fabric blockchain. Section III identifies the dominant and parallelism computing resources of each decomposition phase through primitive operation analysis. Section IV presents a stochastic computation model and an alpha-beta communication model for each decomposition phase in Hyperledger Fabric. Section V describes the implementation details of the client-driven measurement framework and the validation results of the proposed model. Finally, Section VI concludes this paper.

II. RELATED WORK

Section II reviews related work in performance analysis and modeling of Hyperledger Fabric, including throughput and latency. TABLE I summarizes a comparison of the state-of-the-art performance works on Hyperledger Fabric blockchains.

A. Performance Analysis of Hyperledger Fabric Blockchain

Most existing works studied the overall latency and throughput of Hyperledger Fabric service. A typical methodology is to tune configuration parameters, restart the system, and analyze performance metrics. This methodology [61], [70]–[75] is intuitive and allows people to understand the impact of different configuration parameters on the system performance. Related work that predominantly focused on system performance can be classified into four main paradigms: system measurement-based, scalability-based, security-relevant, and blockchain application-relevant.

1) *System Measurement-based*: To characterize the performance of Hyperledger Fabric blockchain, system measurement is an important research method. For example, the usage of Hyperledger Caliper tool was demonstrated to measure the overall throughput of Hyperledger Fabric service [26]. The impacts of different database systems on the performance of Hyperledger Fabric were characterized in [48], where the authors ran a GoLevelDB database and found that GoLevelDB's data compression could significantly impact Hyperledger Fabric's performance. The effects of endorsement policy on the system performance of Hyperledger Fabric were explored in [27], where the authors aimed to streamline transaction validation processes and to bridge the gap between Hyperledger Fabric's throughput and the demands of large-scale industrial applications by implementing a less-than-half endorsement policy. The effects of multi-version concurrency control (MVCC) were studied in [28], where the authors proposed a solution that reallocates the MVCC from the validation phase to a preceding stage in the transaction flow to enable early detection of conflicting transactions. The effects of continuously expanding ledger were introduced in [45], where the authors proposed an approach to manage the expanding ledger in the context of Hyperledger Fabric for blockchain-based IoT applications. They introduced an algorithm to optimize the ledger size and ensured that the growth of the ledger size is nearly optimal given the incoming transaction proposal rate. Recently, with the rapid development of blockchain technology, more specific applications are utilizing it. Identifying the key performance

TABLE I
A COMPARISON OF THE STATE-OF-THE-ART PERFORMANCE WORKS ON HYPERLEDGER FABRIC BLOCKCHAINS

Works	Overall System Throughput	Overall System Latency	Phase Decomposition (Throughput)			Phase Decomposition (Latency)			Waiting Time	
			The execute phase	The order phase	The validate phase	The execute phase	The order phase	The validate phase	Performance modeling	Reduction strategy
[25]	✓	✓	✗	✗	✗	✓	✓	✓	✗	✗
[42]	✓	✓	✗	✓	✓	✗	✓	✓	✗	✗
[43], [44]	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗
[4], [22], [24]	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
[26]–[28], [39]	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
[45]–[66]	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
[67]	✓	✓	✗	✗	✗	✗	✗	✗	✓	✗
[68], [69]	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Ours	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

parameters for specific scenarios is an important research topic. To propose a set of performance criteria for evaluating Hyperledger Fabric latency performance modeling and to highlight key Hyperledger Fabric parameters that influence latency, many efforts have been made in [24], [47].

2) *Scalability-based*: To improve the scalability of Hyperledger Fabric blockchain, researchers have explored various schemes [60], [65], [76], [77], such as subset validation and sharding. For example, the scalability of the validate phase in Hyperledger Fabric was studied in [42], where the authors found that the validate phase was a performance bottleneck in Hyperledger Fabric, and proposed an improvement scheme called sparse peer to enhance the scalability of the validation phase by validating only a subset of transactions. And a sharding-based scalability method was proposed in [65], where the authors proposed a block alliance consensus mechanism that considered the blockchain impossible triangle in the energy trading scenario by reasonably pursuing decentralization. They achieved infinite scalability via sharding, designed a cross-shard method for transactions between different shards, and implemented the energy trading blockchain and the consensus mechanism on the Hyperledger Fabric platform. Similarly, based on a multi-shard architecture, the authors in [66] proposed a partitioned, parallel, and practicable permissioned blockchain. Their key design was to extend scalability in terms of consensus algorithm protocol, architecture, and scheduling. The blockchain was implemented in Golang across multiple operating systems and evaluated on the Hyperledger Fabric.

3) *Security-relevant*: Different from permissionless blockchain, Hyperledger Fabric is a permissioned blockchain where data is only available to authorized users. To address real-world attacks on blockchains and to provide convenient and secure blockchain service, many security-relevant schemes [57], [78], [79] have been proposed. For example, in [46], the authors addressed the security issue of decreased mission success rate caused by malicious attacks. They selected Hyperledger Fabric as the blockchain platform for data encryption, consensus and reliability calculations, and smart contract deployment. Another work [53] constructed a decentralized, secure, and flexibly expressive crowdsourcing task-matching system robust to misbehaviors based on Hyperledger Fabric blockchain. They demonstrated the desired security properties and superior on-chain and off-chain performance of the Hyperledger Fabric-based system. And, in [54], the authors proposed a privacy-conscious

and publicly verifiable data quality assessment scheme in blockchain-empowered crowdsourcing that was resilient against stronger malicious adversaries. They conducted experiments over Hyperledger Fabric on two real-world datasets, demonstrating satisfactory accuracy, robustness, and on-chain and off-chain performance. Similarly, the authors in [4] proposed a blockchain-assisted industrial data registration and reconstruction management scheme on Hyperledger Fabric. The tamper-proof and distributed storage characteristics of blockchain were utilized to ensure the authenticity and transparency of output errors throughout the reconstruction process. Moreover, in [56], the authors provided a trusted environment for cloud service providers, vertical service providers, and mobile network operators to manage the lifecycle of network services, such as instantiation, scaling, and termination, in a multi-cloud environment. The proposed approach is validated through an experimental scenario using Hyperledger and Quorum to measure various performance metrics related to service orchestrator-related instantiation and credential verification metrics to ensure improved feasibility, scalability, reliability, and performance.

4) *Blockchain Application-relevant*: The development of blockchain technology has led to an increasing implementation and deployment of blockchain applications. The performance analysis of these blockchain-based applications [80], [81] poses new challenges for researchers. For example, the authors in [68] implemented a consortium blockchain network utilizing the Hyperledger Fabric framework complemented by the Hyperledger Composer toolset. They demonstrated that the implemented Hyperledger Fabric-based system exhibited strong performance metrics with satisfactory throughput and latency for healthcare applications. Moreover, the authors in [52] proposed a Hyperledger-based voting system integrating with Django and React frameworks. Their implementation of the voting system offered a user-friendly Android application for voters and a separate administration portal for efficient election management, and demonstrated commendable performance and functionality.

Although the overall performance of most related work, i.e., latency and throughput, provides an end-user with sufficient information when invoking or querying a transaction, it is not informative for a system deployer to deploy the Hyperledger Fabric service on a cluster to meet specific performance goals, except for understanding each decomposition phase of Hyperledger Fabric. Currently, there are few related works on the

performance characteristics of each decomposition phase, and a measurement method and framework for each decomposition phase remains lacking. Thus, based on the current state-of-the-art system architecture of execute-order-validate (EOV) phases [20], [22], we propose a measurement framework to characterize the performance of each decomposition phase.

B. Performance Modeling of Hyperledger Fabric Blockchain

Performance modeling is important because it helps to characterize the performance of Hyperledger Fabric service quantitatively. Recent works [25], [51], [69] on the performance models of Hyperledger Fabric blockchain can be classified into three main types: queue modeling-based, scheduling algorithm-based, and machine learning-relevant.

1) *Queue Modeling-based*: To analyze the fluctuation of transaction arrival times and service times, many recent works [82]–[84] based on queueing theory have been proposed. For example, the authors in [51] provided a queue model to calculate the transaction latency under various network configurations, such as batch size and block interval, and proposed an $M/M/1$ queue to model the PBFT-based ordering service in a solo mode, i.e., with one ordering service node. The authors in [44] analyzed the security properties of Hyperledger Fabric service and the impact of delaying endorsement messages in consortium blockchain protocols. And in [43], the authors proposed a model to analyze the performance of a Hyperledger-based system by using generalized stochastic Petri nets. They decomposed a transaction flow into multiple phases, provided a simulation-based approach to obtain the system latency and throughput with a specific arrival rate, and analyzed the impact of different configurations of ordering service on system performance. Similarly, the authors in [63], used a stochastic Petri net model to analyze transaction flows in Hyperledger Fabric networks, achieving a high confidence interval for response times. Their performance model enabled administrators to assess the impact of system changes on resource utilization. Moreover, the authors in [62] proposed a performance-compatible modeling method for Fabric using queueing theory, which considered the limited transaction pool and the situation where node groups were attacked. They established a model of the transaction process in the Fabric network, solved the system stationary equation, and obtained analytical expressions for performance indicators such as system throughput, system steady-state queue length, and system average response time.

2) *Scheduling Algorithm-based*: To fast process transactions and to provide convenient blockchain transaction clearing services, researchers have proposed various scheduling algorithms [55], [58], [61]. For example, the authors in [69] proposed a transaction batch processing scheduling algorithm for multi-channel Fabric networks based on Lyapunov optimization theory. Their algorithm maximized the consensus efficiency of the system, ensured the minimum transaction accumulation, and provided stability conditions and performance for the system under transaction batch processing. And the authors in [67], proposed an analytical framework for relay chain performance. They calculated the relay chain with practical BFT consensus and developed a method to arrive at the

computational forms of two essential performance descriptors: system throughput and cross-chain transaction confirmation delay. Using their performance model, they judged whether the relay chain was overloaded and eliminated the overload state by tuning the parameters. Moreover, the authors in [61] made use of a hierarchical approach to the Hyperledger Fabric Blockchain on the IoT network so that they could control the size of the ledger that each sub-network would require. They resolved the ledger size and cost issues that plague many of the existing solutions in the literature without compromising on the main benefits of the Blockchain, and enhanced the integrity, authorization, and authentication of the communication between the IoT devices in the network.

3) *Machine Learning-relevant*: To extract efficient parameter configuration information, many related works have attempted to use machine learning methods [59], [64], [85]. An advantage of the machine learning method is its high fitting accuracy, but a disadvantage is that it requires a large amount of system-level data and that the system model often remains a black box. For example, the authors in [50] applied a temporal convolution neural network to predict the transaction arrival rate of the blockchain and proposed an ensemble learning-based method and a meta-learning-based method to train a blockchain performance prediction model and to dynamically tune the configuration parameters of the blockchain to optimize the blockchain performance. The authors in [25] selected Hyperledger Fabric v2.0 as the research target. They focused on the latency of the ordering phase that accounts for the highest proportion of latency in the execute-order-validate architecture. They modeled the optimization of blockchain experimentation as a Markov decision process and introduced a dynamic parameter configuration method grounded in a deep reinforcement learning approach to reduce latency. Similarly, the authors in [64] considered the latency challenge in Hyperledger Fabric by proposing a reinforcement learning-based dynamic endorsement mechanism. Their model learned from past transaction patterns and system states to predict the optimal number of endorsers needed for each transaction, and to reduce transaction latency. Moreover, the authors in [49] studied a blockchain-enhanced architecture for training online traffic prediction models using a Federated Learning approach. Their approach enables decentralized model training at the edge of the intelligent transportation system network, and experiments on Hyperledger Fabric used dynamically collected arterial traffic data shards as a case study to evaluate online learning performance.

However, most related work focuses on the high accuracy of latency and throughput of the Hyperledger Fabric service under normal cases, and no related work considers the worst case, i.e., the order phase of Hyperledger Fabric encountering unfit block-setting parameters. The worst-case analysis of the order phase is important. On one hand, the order phase addresses the consensus challenge in Hyperledger Fabric blockchains [86]–[90]. On the other hand, it brings a waiting time to a transaction in the order phase. Moreover, waiting time would dominate a transaction's latency in the order phase when the worst case occurs. Thus, we aim to reduce waiting time when it dominates a transaction's latency in the order phase under

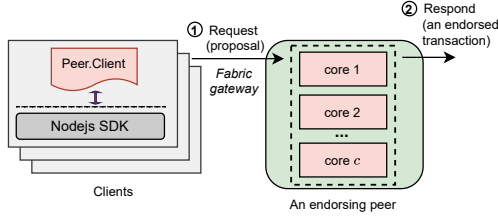


Fig. 2. An illustration of the dominant and parallel computing resources in the execute phase, where an endorsing peer has c CPU cores.

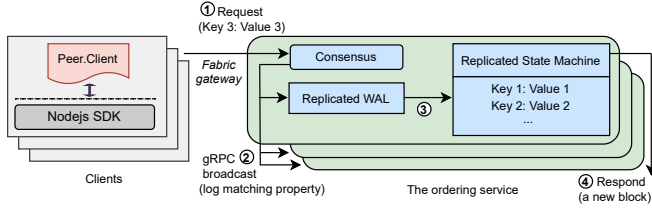


Fig. 3. An illustration of the dominant and parallel computing resources in the order phase, where an ordering service has $k = 3$ OSNs, i.e., one OSN leader and two OSN followers.

unfit block-setting parameters.

III. SYSTEM DESCRIPTION

Section III decomposes the system into three phases and identifies the dominant and parallel computing resources for each decomposition phase.

A. The Execute Phase

Dominant and Parallel Computing Resources in the Execute Phase. The primitive operations of the execute phase involve two parts: proposal verification and transaction simulation. Specifically, upon receiving a transaction proposal, the execute phase completes proposal verification (i.e., the transaction proposal is well-formed, it has not been submitted before, the client's signature is valid, and the client is authorized to submit a transaction proposal). After proposal verification, the execute phase completes the transaction simulation (i.e., it takes the transaction proposal as an input argument to invoke the user chaincode and get the transaction results, and then, the execute phase uses an elliptic curve digital signature algorithm (ECDSA-256) to make a digital signature and return the transaction results and the signature to the client).

Fig. 2 illustrates the dominant and parallel computing resources in the execute phase. In step ①, the client initializes a transaction proposal and submits it to an endorsing peer via the Fabric Gateway in the execute phase. In step ②, the endorsing peer simulates and endorses the proposal and passes back the response to the client. Note that the execute phase handles these primitive operations concurrently on multiple CPU cores. It implies that the service times of transactions can be parallelized in the execute phase.

B. The Order Phase

Dominant and Parallel Computing Resources in the Order Phase. The primitive operations of the order phase involve two

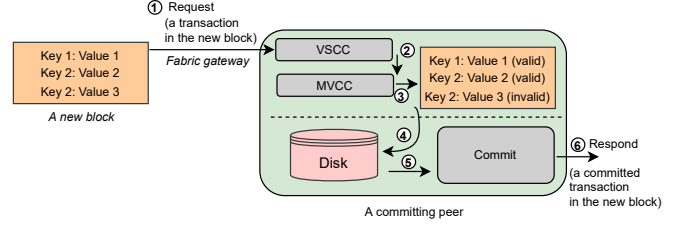


Fig. 4. An illustration of the dominant and parallel computing resources in the validate phase, where there is a committing peer. Consider that the ordering service creates a new block with three transactions. The first transaction writes key 1 with value 1, the second one writes key 2 with value 2, and the third one writes key 2 with value 3.

parts: the communication latency between the OSN leader and the OSN followers in the Raft protocol and the waiting time when batching a new block.

Fig. 3 illustrates the dominant and parallel computing resources in the order phase. In step ①, the client submits transactions to the local memory of the OSN leader over the network. In step ②, the OSN leader concurrently broadcasts these new transactions to the local memory of the OSN followers over the network while ensuring that at least a majority of OSN followers maintain the same sequence of these new transactions. In steps ③, ④, the OSN leader puts the transactions from a preordered sequence into a new block, commits the new block to local memory storage instead of disk storage, replies to peers, and forwards the new block to the OSN followers. Note that a block is technically a tiny offset pointer since most OSN followers have already successfully maintained a preordered sequence of new transactions.

The second primitive operation of the order phase is the block cutter on the OSN leader. The block cutter on the OSN leader brings a waiting time to a transaction. Specifically, the Raft consensus protocol handles data from a transaction level, while the Raft-based ordering service handles data from a block level. It implies that a transaction that quickly achieves a Raft consensus may not be put into a new block immediately, and it waits until all transactions in the block achieve the Raft consensus. Therefore, it needs to wait until enough transactions arrive and a new block is created. And there are two block-setting parameters that determine the waiting time: *BatchTimeout* and *BatchSize*. The *BatchTimeout* determines the time to wait until creating a new block, and the *BatchSize* determines the total number of transactions and the total transaction size of a new block. The ordering service generates a new block if either of the two block-setting parameters is satisfied.

C. The Validate Phase

Dominant and Parallel Computing Resources in the Validate Phase. The primitive operations of the validate phase involve two parts: the validate operation and the commit operation. As shown in Fig. 4, there are three transactions within a new block. In step ①, the block is transmitted to the peer in the validate phase. Upon receiving a new block, the peer in step ② validates each transaction within the block and ensures that

TABLE II
SYMBOL DEFINITIONS

Variable	Description	Units
ζ	The number of clients	Dimensionless
c	The number of CPU cores of an endorsing peer in the execute phase	Dimensionless
k	The number of OSNs in the order phase where k is an odd number	Dimensionless
CV	The coefficient of variation	Dimensionless
m	The transaction size, i.e., 3 KB	KB
α	The constant communication overhead, where it takes $\alpha = 10$ ms in a cluster	Millisecond
β	The effective network bandwidth	Mbps
λ^e	The transaction arrival rate of an endorsing peer with c CPU cores in the execute phase	TPS
μ^e	The transaction service rate of an endorsing peer with c CPU cores in the execute phase	TPS
ρ^e	The utilization of an endorsing peer with c CPU cores in the execute phase	Dimensionless
ψ^e	The throughput of an endorsing peer with c CPU cores in the execute phase	TPS
$T^e = \{\tau_i^e; i \in \mathbb{Z}^+\}$	The τ_i^e is the overall latency of the i -th transaction in the execute phase	Second
$T_{c2e}^e = \{\tau_{i,c2e}^e; i \in \mathbb{Z}^+\}$	The $\tau_{i,c2e}^e$ is the communication latency of the i -th transaction from client to endorsing peer	Second
$T_{e2c}^e = \{\tau_{i,e2c}^e; i \in \mathbb{Z}^+\}$	The $\tau_{i,e2c}^e$ is the communication latency of the i -th transaction from endorsing peer to client	Second
$T_{comm}^e = \{\tau_{i,comm}^e; i \in \mathbb{Z}^+\}$	The $\tau_{i,comm}^e$ is the communication latency of the i -th transaction in the execute phase	Second
$T_s^e = \{\tau_{i,s}^e; i \in \mathbb{Z}^+\}$	The $\tau_{i,s}^e$ is the service time of the i -th transaction in the execute phase	Second
$T_q^e = \{\tau_{i,q}^e; i \in \mathbb{Z}^+\}$	The $\tau_{i,q}^e$ is the queueing latency of the i -th transaction in the execute phase	Second
λ^r	The transaction arrival rate of the OSN leader in the order phase	TPS
μ^r	The transaction service rate of the ordering service with k OSNs in the order phase	TPS
ρ^r	The utilization of the ordering service with k OSNs in the order phase	Dimensionless
ψ^r	The throughput of the ordering service with k OSNs in the order phase	TPS
$T^r = \{\tau_i^r; i \in \mathbb{Z}^+\}$	The τ_i^r is the overall latency of the i -th transaction in the order phase	Second
$T_{c2l}^r = \{\tau_{i,c2l}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,c2l}^r$ is the communication latency of the i -th transaction from client and to OSN leader	Second
$T_{l2f}^r = \{\tau_{i,l2f}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,l2f}^r$ is the communication latency of the i -th transaction from OSN leader to followers	Second
$T_{comm}^r = \{\tau_{i,comm}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,comm}^r$ is the communication latency of the i -th transaction in the order phase	Second
$T_s^r = \{\tau_{i,s}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,s}^r$ is the service time of the i -th transaction in the order phase	Second
$T_w^r = \{\tau_{i,w}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,w}^r$ is the waiting time of the i -th transaction when batching a block in the order phase	Second
$T_q^r = \{\tau_{i,q}^r; i \in \mathbb{Z}^+\}$	The $\tau_{i,q}^r$ is the queueing latency of the i -th transaction in the order phase	Second
λ^v	The transaction arrival rate of a committing peer in the validate phase	TPS
μ^v	The transaction service rate of a committing peer in the validate phase	TPS
ρ^v	The utilization of a committing peer in the validate phase	Dimensionless
ψ^v	The throughput of a committing peer in the validate phase	TPS
$T^v = \{\tau_i^v; i \in \mathbb{Z}^+\}$	The τ_i^v is the overall latency of the i -th transaction in the validate phase	Second
$T_{comm}^v = \{\tau_{i,comm}^v; i \in \mathbb{Z}^+\}$	The $\tau_{i,comm}^v$ is the communication latency of the i -th transaction in the validate phase	Second
$T_s^v = \{\tau_{i,s}^v; i \in \mathbb{Z}^+\}$	The $\tau_{i,s}^v$ is the service time of the i -th transaction in the validate phase	Second
$T_q^v = \{\tau_{i,q}^v; i \in \mathbb{Z}^+\}$	The $\tau_{i,q}^v$ is the queueing latency of the i -th transaction in the validate phase	Second

the endorsement policy is fulfilled. Then, the multi-version concurrency control (MVCC) chaincode handles concurrent transactions in step ③. Concurrent transactions with read-write conflicts are marked as invalid. Both valid and invalid transaction data are stored in the blockchain and in the local database. The state of an invalid transaction is not applied to the global state of the Hyperledger Fabric blockchain. Hyperledger Fabric applies validation to the read set to prevent double-spending attacks. In step ④ and ⑤, the committer stores block data to the local database and applies the state of each valid transaction to the global state. In step ⑥, the committer notifies the client about the valid or invalid state status.

IV. SYSTEM MODELING

Section IV presents a stochastic computation model and an alpha-beta communication model. TABLE II summarizes the symbol definitions of our system model.

A. The Execute Phase

Throughput of an Endorsing Peer. Assume the transaction arrival from a client follows a Poisson distribution, the service

time is deterministic, and an endorsing peer serves ζ clients. Let the transaction arrival rate of the client i be λ_i^e transactions per second. Therefore, the execute phase has a transaction arrival rate $\lambda^e = \zeta \lambda_i^e$ due to the merging property [91], [92] of Poisson processes.

Consider an endorsing peer with c CPU cores as an $M/D/c$ queue because the execute phase's endorsing operations can be parallelized. Therefore, we have a linear model of the throughput of an endorsing peer with c CPU cores as follows,

$$\psi^e = \sum_{i=1}^c \psi_i^e = c \psi_i^e; \psi_i^e = \mu^e \rho^e \quad (1)$$

where the service rate of the CPU core i of the endorsing peer is denoted by ψ_i^e and $\rho^e = \frac{\lambda^e}{c \mu^e}$.

Expected Latency in the Execute Phase. Consider the overall latency of a transaction in the execute phase is a random variable $T^e = \{\tau_i^e; i \in \mathbb{Z}^+\}$ where τ_i^e is the overall latency of the i -th transaction in the execute phase. T^e consists of three parts: the communication latency of a transaction in the execute phase denoted by a random variable $T_{comm}^e = \{\tau_{i,comm}^e; i \in \mathbb{Z}^+\}$ where $\tau_{i,comm}^e$ is the communication latency of the i -th transaction in the execute phase, the service

time of a transaction in the execute phase denoted by a random variable $T_s^e = \{\tau_{i,s}^e; i \in \mathbb{Z}^+\}$ where $\tau_{i,s}^e$ is the service time of the i -th transaction in the execute phase, the queueing latency of a transaction in the execute phase denoted by a random variable $T_q^e = \{\tau_{i,q}^e; i \in \mathbb{Z}^+\}$ where $\tau_{i,q}^e$ is the queueing latency of the i -th transaction in the execute phase. Therefore, we have the expected latency of a transaction in the execute phase as follows,

$$\begin{aligned} \mathbb{E}[T^e] &= \mathbb{E}[T_{comm}^e] + \mathbb{E}[T_s^e] + \mathbb{E}[T_q^e] \\ &= 2\alpha + \frac{2m\lambda^e}{\beta^e} + \frac{1}{\mu^e} + \frac{1}{2}(1 + f(c)g(\rho^e))\mathbb{E}\left[T_q^{M/M/c}\right] \end{aligned} \quad (2)$$

where, first, after applying the alpha-beta communication model, the expected communication latency from the client to the endorsing peer is $\mathbb{E}[T_{c2e}^e] = \alpha + \frac{m'\lambda^e}{\beta_{c2e}^e}$, and the expected communication latency from the endorsing peer to the client is $\mathbb{E}[T_{e2c}^e] = \alpha + \frac{m'\lambda^e}{\beta_{e2c}^e}$. In the real world, the transaction size (i.e., around 3KB) minus the size of a 256-bit ECDSA signature equals the transaction proposal size, and we approximate $m = m'$. Besides, the bi-directional communication between the client and the endorsing peer (i.e., from the client to the endorsing peer and from the endorsing peer to the client) utilizes the same gRPC communication protocol. Therefore, we approximate the expected communication latency of a transaction in the execute phase as $\mathbb{E}[T_{comm}^e] = \mathbb{E}[T_{c2e}^e] + \mathbb{E}[T_{e2c}^e] = 2\alpha + \frac{2m\lambda^e}{\beta^e}$, where α is a constant communication overhead, and β^e is the effective bandwidth between the endorsing peer and the clients in the execute phase. Second, the expected service time of the execute phase is modeled as $\mathbb{E}[T_s^e] = \frac{1}{\mu^e}$, where μ^e means the maximum throughput of a single CPU core in the execute phase. Third, the queueing latency of the execute phase is modeled as an $M/D/c$ queue approximation [93], [94].

B. The Order Phase

Throughput of an OSN Leader. Assume the transaction arrival of the order phase follows a General distribution, and the service time also follows a General distribution. The transaction arrival rate of the order phase $\lambda^r = \psi^e$ because the output of the execute phase becomes the input of the order phase. Denote the throughput of the ordering service with k OSNs by ψ^r . The process of transaction ordering in the OSN leader is modeled as a $G/G/1$ queue because its job is to sequence transactions from the clients, and the service times of sequencing these transactions are not parallelized. Therefore, we have the throughput of the OSN leader with i OSN followers as follows,

$$\psi^r = \psi_i^r, i = 3, 5, 7 \dots, k-1; \psi^r = \mu^r \rho^r \quad (3)$$

Expected Latency in the Order Phase. Consider the overall latency of a transaction in the order phase is a random variable $T^r = \{\tau_i^r; i \in \mathbb{Z}^+\}$ where τ_i^r is the overall latency of the i -th transaction in the order phase. T^r consists of five parts: the communication latency of a transaction from the client to the OSN leader denoted by a random variable $T_{c2l}^r = \{\tau_{i,c2l}^r; i \in \mathbb{Z}^+\}$ where $\tau_{i,c2l}^r$ is the i -th transaction's

communication latency from the client to the OSN leader, the communication latency of a transaction from the OSN leader to the OSN followers denoted by a random variable $T_{l2f}^r = \{\tau_{i,l2f}^r; i \in \mathbb{Z}^+\}$ where $\tau_{i,l2f}^r$ is the i -th transaction's communication latency from the OSN leader to the OSN followers, the service time of a transaction in the OSN leader denoted by a random variable $T_s^r = \{\tau_{i,s}^r; i \in \mathbb{Z}^+\}$ where $\tau_{i,s}^r$ is the i -th transaction's service time, the waiting time of a transaction in the ordering service denoted by a random variable $T_w^r = \{\tau_{i,w}^r; i \in \mathbb{Z}^+\}$ where $\tau_{i,w}^r$ is the i -th transaction's waiting time, the queueing latency of a transaction in the ordering service denoted by a random variable $T_q^r = \{\tau_{i,q}^r; i \in \mathbb{Z}^+\}$ where $\tau_{i,q}^r$ is the i -th transaction's queueing latency. Therefore, we have the expected latency of a transaction in the order phase as follows,

$$\begin{aligned} \mathbb{E}[T^r] &= \mathbb{E}[T_{c2l}^r] + \mathbb{E}[T_{l2f}^r] + \mathbb{E}[T_s^r] + \mathbb{E}[T_w^r] + \mathbb{E}[T_q^r] \\ &= \alpha + \frac{m\lambda^r}{\beta_{c2l}^r} + \alpha + \frac{(n-1)m\lambda^r}{\beta_{l2f}^r} + \frac{1}{\mu^r} \\ &\quad + \min\left(\frac{BatchTimeout}{2}, \frac{\Delta t}{2}\right) \\ &\quad + \left(\frac{\rho^r}{1-\rho^r}\right) \left(\frac{(CV_a^r)^2 + (CV_s^r)^2}{2}\right) \frac{1}{\mu^r} \end{aligned} \quad (4)$$

$$\frac{\lambda^r \Delta t}{\min \Delta t} \geq BatchSize \quad (5)$$

where, first, after applying the alpha-beta communication model, the expected communication latency from the client to the OSN leader is $\mathbb{E}[T_{c2l}^r] = \alpha + \frac{m\lambda^r}{\beta_{c2l}^r}$, where λ^r is the transaction arrival rate, β_{c2l}^r is the effective bandwidth from the client to the OSN leader. Second, the expected communication latency from the OSN leader to the $k-1$ OSN followers is $\mathbb{E}[T_{l2f}^r] = \alpha + \frac{(k-1)m\lambda^r}{\beta_{l2f}^r}$, where λ^r is the given transaction arrival rate, β_{l2f}^r is the effective bandwidth from the OSN leader to the $k-1$ OSN followers. Third, the expected service time of the OSN leader is modeled as $\mathbb{E}[T_s^r] = \frac{1}{\mu^r}$. Fourth, the waiting time of a transaction T_w^r has two cases. For case ①: the *BatchTimeout* condition is satisfied, and the expected waiting time of a transaction is modeled as $\mathbb{E}[T_w^r] = \frac{BatchTimeout}{2}$. For case ②: the *BatchSize* condition is satisfied. The expected waiting time of a transaction is modeled as $\mathbb{E}[T_w^r] = \frac{\Delta t}{2}$, where Δt satisfies the condition that during a period of Δt , the number of arriving transactions with the transaction arrival rate equals to *BatchSize* and the second condition is satisfied. Fifth, the process of transaction ordering in the OSN leader is modeled as a $G/G/1$ queue approximation [95], [96].

C. The Validate Phase

Throughput of a Committing Peer. Assume the transaction arrival of the validate phase follows a General distribution, and the service time also follows a General distribution. The transaction arrival rate of the validate phase $\lambda^v = \psi^r$ because the output of the order phase becomes the input of the validate phase. Consider a committing peer in the validate phase as an

anchor peer, and there is no extra hop between the ordering service and the committing peer. Denote the throughput of a committing peer by ψ^v . The process of a committing peer in the validate phase is modeled as a $G/G/1$ queue because the validation of a block depends on the committed result of its preceding block to prevent double-spending attacks; therefore, the service times are not parallelized. We have the throughput of a committing peer as follows,

$$\psi^v = \mu^v \rho^v = \frac{dR_{disk}\rho^v}{m} \quad (6)$$

where d is the written data size per disk IO and R_{disk} is the number of IOs per second. For an HDD disk, the number of IOs per second is $\frac{1}{R_{disk}} = W_{seek} + 1/\text{IOs}$, where W_{seek} represents the seek latency per IO. For an SSD disk, the number of IOs per second is $\frac{1}{R_{disk}} = 1/\text{IOs}$ with no seek latency per IO.

Expected Latency in the Validate Phase. Consider the overall latency of a transaction in the validate phase is a random variable $T^v = \{\tau_i^v; i \in \mathbb{Z}^+\}$ where τ_i^v is the overall latency of the i -th transaction in the validate phase. T^v consists of three parts: the communication latency of a transaction from the ordering service to the committing peer denoted by a random variable $T_{comm}^v = \{\tau_{i,comm}^v; i \in \mathbb{Z}^+\}$ where $\tau_{i,comm}^v$ is the i -th transaction's communication latency, the service time of a transaction in the validate phase denoted by a random variable $T_s^v = \{\tau_{i,s}^v; i \in \mathbb{Z}^+\}$ where $\tau_{i,s}^v$ is the i -th transaction's service time, the queueing latency of a transaction in the validate phase denoted by a random variable $T_q^v = \{\tau_{i,q}^v; i \in \mathbb{Z}^+\}$ where $\tau_{i,q}^v$ is the i -th transaction's queueing latency. Therefore, we have the expected latency of a transaction in the validate phase as follows,

$$\begin{aligned} \mathbb{E}[T^v] &= \mathbb{E}[T_{comm}^v] + \mathbb{E}[T_s^v] + \mathbb{E}[T_q^v] \\ &= \alpha + \frac{m\lambda^v}{\beta^v} + \frac{1}{\mu^v} + \left(\frac{\rho^v}{1 - \rho^v} \right) \left(\frac{(CV_a^v)^2 + (CV_s^v)^2}{2} \right) \frac{1}{\mu^v} \end{aligned} \quad (7)$$

where, first, after applying the alpha-beta communication model, the expected communication latency from the ordering service to the committing peer is $\mathbb{E}[T_{comm}^v] = \alpha + \frac{m\lambda^v}{\beta^v}$, where λ^v is the transaction arrival rate, β^v is the effective bandwidth from the ordering service to the committing peer. Second, the expected service time of the validate phase is modeled as $\mathbb{E}[T_s^v] = \frac{1}{\mu^v}$. Third, the expected queueing latency of the validate phase is modeled as a $G/G/1$ queue approximation [95], [96].

V. MEASUREMENT FRAMEWORK AND EVALUATION

Section V presents a client-driven measurement framework for phase decomposition, and the experimental validation of each decomposition phase in Hyperledger Fabric.

A. Measurement Framework

We present the implementation details of the client-driven measurement framework.

Separation of Hyperledger Fabric Service and Client. Hyperledger Fabric system can be divided into two parts: service and client. The service side is the core of the Hyperledger Fabric system and involves several components, including peers, orderers, and smart contracts. The client side is a lightweight member of the Hyperledger Fabric system. A client of a specific peer only needs to own a membership certificate issued by the Fabric CA. The relationship between the service and client sides is that each peer on the service side may have several clients, while each client belongs to a specific peer.

The client and service sides take different jobs. The client side generates new transaction proposals for the endorsing peer and new transactions for the ordering service. The service side handles these new transaction proposals and transactions from the client side. People separate the client from the service side in most real-world applications because transactions typically originate from the client side, not the service side. The communication method between the client side and the service side is as follows: Hyperledger Fabric 2.2 LTS provides a toolkit named Fabric SDK Node.js that enables many essential functions between the client and service sides. For example, the Fabric SDK Node.js allows the client to prepare and submit a new transaction proposal or a transaction to the service side. Upon the transaction proposal endorsed by the endorsing peer or a block is committed by a peer, the service side will notify the client side via a callback function. Thus, based on the functions provided by the Fabric SDK Node.js, our measurement framework can trace the transaction flow between the client and service sides.

Container-based Resource Management. Hyperledger Fabric service is often deployed in a container-based isolated environment because it is convenient for resource management, and the cost of the Docker container can be neglected. Our measurement framework provides a method for identifying the dominant resource in each phase of Hyperledger Fabric through container-based resource management. First, we use the `cpuset` command supplied by Docker Compose v2.4 to control the number of available CPU cores for an endorsing peer during the execute phase. Second, we use the Linux `iftop` to measure the network efficient bandwidth of end-to-end communication. Third, we use the command of `device_write_iops` provided by docker-compose v2.4 and control the disk IOs of a committing peer in the validate phase. Fourth, we utilize the Network Time Protocol (NTP) across all hosts (i.e., containers) for distributed system clock synchronization. And NTP achieves an error of around 2 to 3 milliseconds among all hosts within a cluster (i.e., via connecting to a single standard time server's IP address in practice), which is negligible compared to a transaction's latency.

Stable Requests via Asynchronous & Concurrent Threads. There are two requirements for arrival transactions in Hyperledger Fabric. The first requirement is to generate a large number of stable arriving transactions from the client side to utilize the resource on the service side of Hyperledger Fabric. In practice, a single client usually cannot generate enough transactions to utilize a peer effectively. Thus, we present a measurement framework with multiple clients.

For each client, the implementation is technically based on the Fabric SDK Node.js. For each request, Fabric SDK Node.js launches an asynchronous thread to handle it. It implies that the running threads are asynchronous and that no two threads need to wait for each other. There is a blocking function for each Node.js request thread, because a request must go through all the phases individually and wait for the peer to notify the commit status. Also, each client can utilize multiple CPU cores on a single machine. Thus, we consistently generate numerous arrival transactions by launching multiple clients on several machines, and satisfy the first requirement.

The second requirement is to control the transaction arrival rate of the first phase, i.e., the execute phase in Hyperledger Fabric. We implement the Poisson-based arrival of transactions for the execute phase by introducing an exponentially distributed inter-arrival interval to each request. The overall number of arriving transactions from clients also follows a Poisson distribution due to its merging property. The transaction arrival rate is achieved by adding a specific exponentially-distributed latency to each request. Thus, we achieve a transaction arrival rate for the first phase in Hyperledger Fabric, and satisfy the second requirement.

B. Performance Metrics

We present the performance metrics for each decomposition phase in Hyperledger Fabric service. The performance metrics are derived via a timestamp scheme in which the start and end timestamps of each phase are identified from a transaction-based perspective.

Fig. 1 illustrates the timestamp scheme from a transaction-based perspective. Specifically, we use a realization, i.e., tx_2 , to observe the random variables of each phase in Hyperledger Fabric. Specifically, the execute phase of transaction tx_2 starts at timestamp t_1 when the client prepares the transaction proposal and submits it to the endorsing peer. The order phase of transaction tx_2 starts (i.e., the execute phase of transaction tx_2 ends) at timestamp t_4 when the client submits the transaction to the endorsing service. The validate phase of transaction tx_2 starts (i.e., the order phase of transaction tx_2 ends) at timestamp t_8 when a new block is created. Finally,

the validate phase of transaction tx_2 ends at timestamp t_{10} when the new block is successfully committed. Next, we take transaction tx_2 as an example and calculate the performance metrics of each phase.

In the execute phase, the client submits transaction proposal tx_2 to the endorsing peer at timestamp t_1 . The endorsing peer receives the transaction proposal at timestamp t_2 and replies to the client at timestamp t_3 . Finally, the client receives the reply at timestamp t_4 . Therefore, the expected communication latency of a transaction spent between the client and the endorsing peer $\mathbb{E}[T_{comm}^e]$, and the expected latency of a transaction spent in the execute phase $\mathbb{E}[T^e]$ are as follows,

$$\mathbb{E}[T_{comm}^e] = \mathbb{E}[t_2 - t_1 + t_4 - t_3]; \mathbb{E}[T^e] = \mathbb{E}[t_4 - t_1] \quad (8)$$

In the order phase, the client submits transaction tx_2 to the OSN leader at timestamp t_4 . The OSN leader receives the transaction at timestamp t_5 and then broadcasts the transaction tx_2 to $k - 1$ OSN followers. And a consensus among k OSNs on transaction tx_2 is achieved at timestamp t_6 . But the *BatchSize* condition is not satisfied yet, and transaction tx_2 waits for the block cutter for $t_8 - t_7$ seconds until a new block is created by the OSN leader at timestamp t_8 . Therefore, the expected communication latency of a transaction spent from the client to the OSN leader $\mathbb{E}[T_{c2l}^r]$, the expected communication latency of a transaction spent from the OSN leader to the OSN followers $\mathbb{E}[T_{l2f}^r]$, and the expected latency of a transaction spent in the order phase $\mathbb{E}[T^r]$ are as follows,

$$\begin{aligned} \mathbb{E}[T_{c2l}^r] &= \mathbb{E}[t_5 - t_4]; \mathbb{E}[T_{l2f}^r] = \mathbb{E}[t_6 - t_5]; \\ \mathbb{E}[T^r] &= \mathbb{E}[t_8 - t_4] \end{aligned} \quad (9)$$

In the validate phase, the ordering service creates and sends a new block at timestamp t_8 , and the committing peer receives the block at timestamp t_9 . And the block is committed at timestamp t_{10} . Therefore, the expected communication latency of a transaction spent from the ordering service to the committing peer in the validate phase $\mathbb{E}[T_{comm}^v]$, and the expected latency of a transaction spent in the validate phase $\mathbb{E}[T^v]$ are as follows,

$$\mathbb{E}[T_{comm}^v] = \mathbb{E}[t_9 - t_8]; \mathbb{E}[T^v] = \mathbb{E}[t_{10} - t_8] \quad (10)$$

C. Experiment Environments

We conduct experiments in two different environments regarding different clusters.

TABLE III shows the experiment environments of two different clusters. In cluster one, we assign seven machines for seven clients, one machine for one peer, and fifteen machines for fifteen ordering service nodes. Each machine hosts a 4-core Intel i7 CPU of 3.40 GHz, 8 GB of memory, and a 1 TB hard disk. All machines are connected to a 1 Gbps Ethernet network. In cluster two, we assign seven machines for seven clients, one machine for one peer, and fifteen machines for fifteen ordering service nodes. Each machine hosts an Intel Platinum 8269 CY 8-vCPU of 3.2 GHz, 16 GB memory, and

TABLE III
EXPERIMENT ENVIRONMENTS

Configuration	Cluster 1	Cluster 2 (Alibaba)
CPU Model	Intel i7 3.4 GHz	Intel Platinum 8269 CY 3.2 GHz
Memory Size	8 GB	16 GB
Disk Type	HDD	SSD
Inbound Bandwidth	1 Gbps	10 Gbps

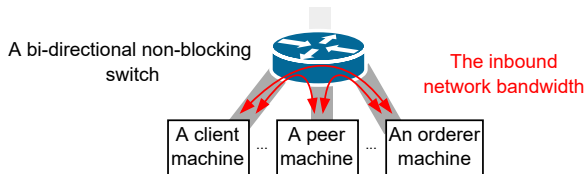


Fig. 5. A network topology where all machines in a cluster are connected via a bi-directional non-blocking switch.

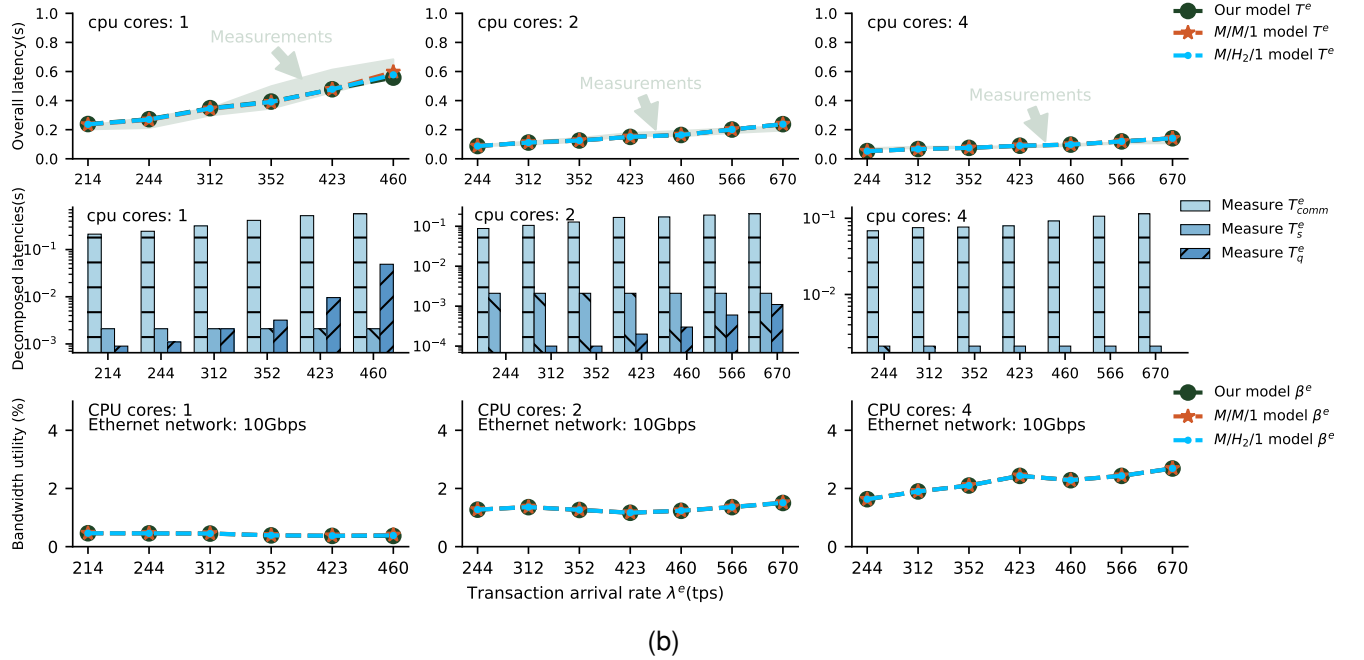
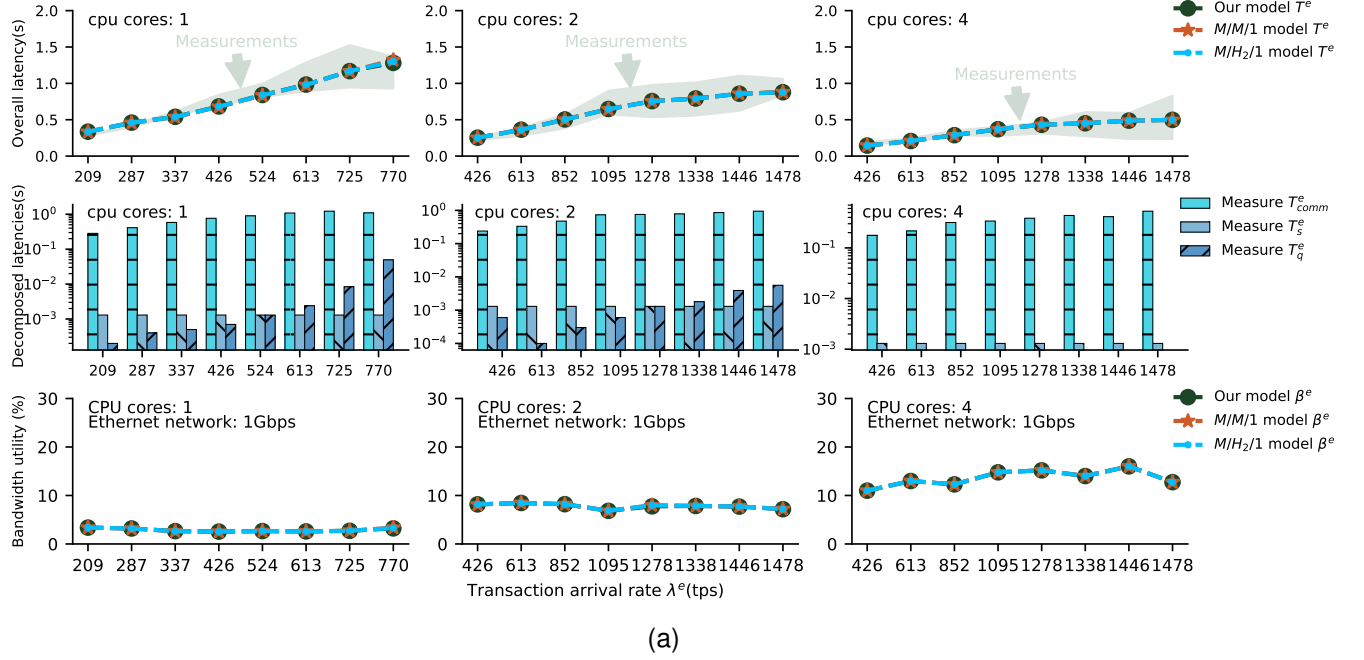


Fig. 6. The effects of CPU cores on latency (T^e), decomposed latencies (T_{comm}^e , T_s^e , T_q^e), and bandwidth utilization (β^e) in the execute phase, where $T^e = T_{comm}^e + T_s^e + T_q^e$. There is an endorsing peer with $c = 1, 2, 4$ CPU core(s), respectively. The remaining machines are seven clients and three ordering service nodes. (a) In cluster one, all machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 1 Gbps. (b) In cluster two, all machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 10 Gbps.

a 40 GB SSD disk. All machines are connected to a 10 Gbps Ethernet network.

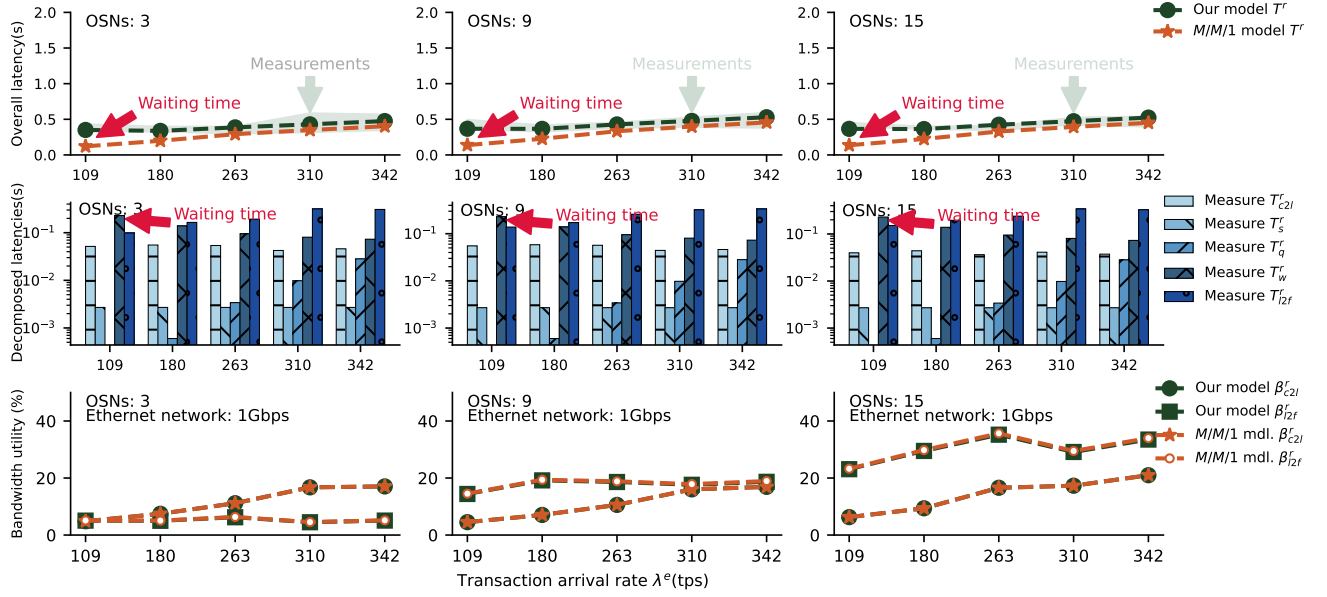
Fig. 5 shows the network topology of each cluster, where all machines of each cluster are directly connected via a bi-directional non-blocking switch.

D. Accuracy and Comparison

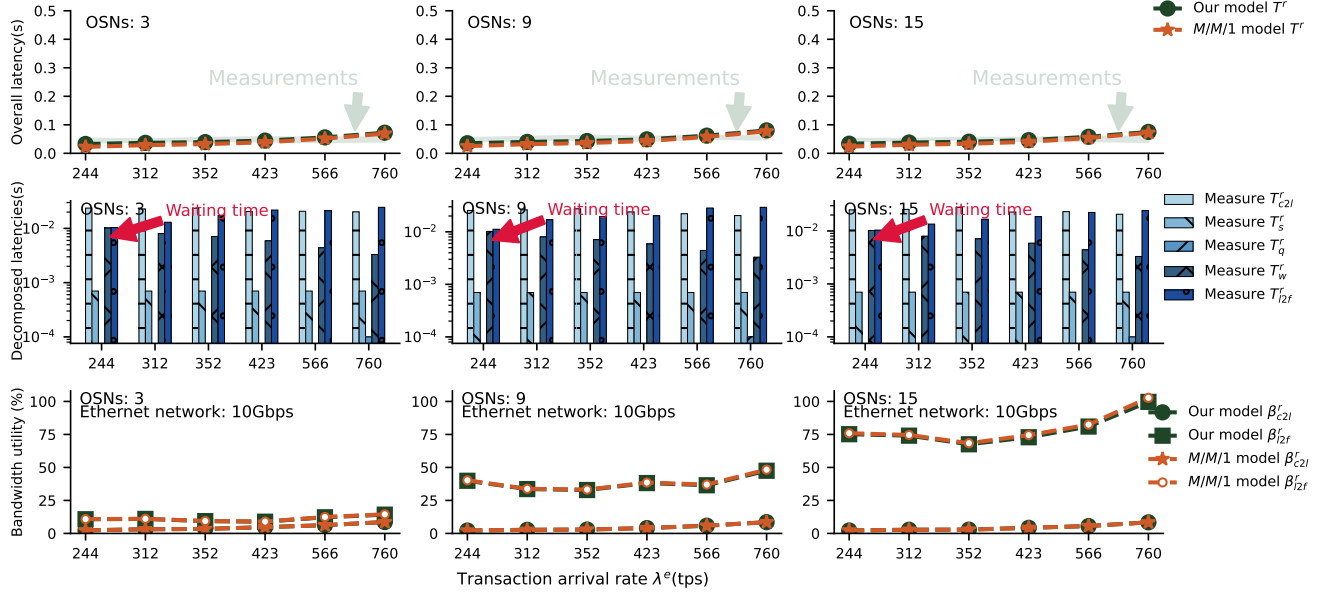
We validate the performance of each decomposition phase, i.e., the execute phase, the order phase, and the validate phase in our proposed model and compare it with recent related

performance models [44], [51]. In particular, in the order phase, we validate our proposed model using two cases: the worst case and the other case. The worst case means waiting time dominates a transaction's latency in the order phase due to unfit block-setting parameters, and the other case means waiting time does not dominate a transaction's latency in the order phase.

In the execute phase, the experiment results illustrate that both our performance model and comparison models, i.e., $M/M/1$ queue [51], $M/H_2/1$ queue [44] well validate the



(a)



(b)

Fig. 7. The effects of OSNs on latency (T^r), decomposed latencies (T_{c2l}^r , T_{l2f}^r , T_w^r , T_s^r , T_q^r), and bandwidth utilization (β_{c2l}^r , β_{l2f}^r), where $T^r = T_{c2l}^r + T_{l2f}^r + T_w^r + T_s^r + T_q^r$. There is an ordering service with $k = 3, 9, 15$ OSNs, respectively. The remaining machines are seven clients and one peer. (a) In cluster one, the *BatchSize* is 50, and the *BatchTimeout* is 1. All machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 1 Gbps. (b) In cluster two, the *BatchSize* is 5, and the *BatchTimeout* is 1. All machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 10 Gbps.

overall latency in the execute phase, as shown in Fig. 6. (a) and (b). Moreover, the decomposed latencies in the execute phase, as shown in Fig. 6. (a) and (b), illustrate that the communication latency between the client and the endorsing peer dominates the overall latency in the execute phase. And an endorsing peer in the execute phase achieves scalable throughput with the number of CPU cores due to parallel service times among these CPU cores, as shown in the bandwidth utility of Fig. 6. (a) and (b).

In the order phase, the experiment results illustrate that for

the worst case, our performance model performs more accurately than the comparison $M/M/1$ models [44], [51] without considering waiting time, when waiting time dominates a transaction's latency in the order phase, as shown in the overall latency of Fig. 7. (a). And, the experiment results illustrate that unfit block-setting parameters would lead to a phenomenon that waiting time dominates a transaction's latency in the order phase, as shown in the decomposed latencies of Fig. 7. (a). For the other cases, both our performance model and comparison models [44], [51] well validate the measurement results as

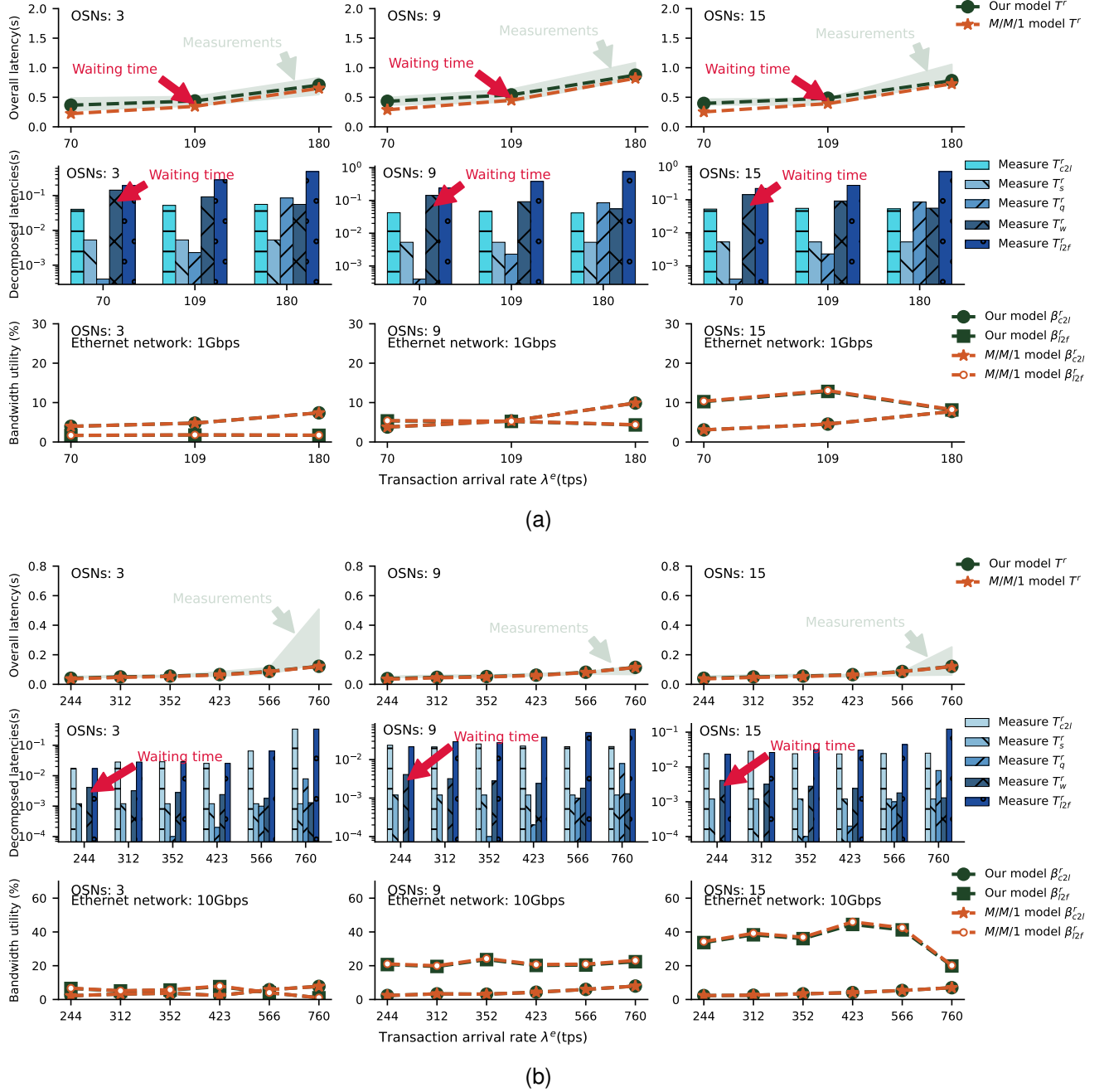


Fig. 8. (a) In cluster one, the significant effects of reducing the *BatchSize* from 50 to 20. (b) In cluster two, the effects of reducing the *BatchSize* from 5 to 2.

shown in the overall latency of Fig. 7. (b). Besides that, the communication latency, particularly that of the Raft protocol, in most cases dominates the overall latency of a transaction spent in the order phase, as shown in the decomposed latencies of Fig. 7. (a) and (b). Finally, the Raft-based ordering service demonstrates good scalability with the number of ordering service nodes due to concurrent processing between the OSN leader and its followers, as shown in the bandwidth utility of Fig. 7. (a) and (b).

In the validate phase, the experiment results illustrate that both our performance model and related comparison models, i.e., $M/M/1$ queue [51], $M/E_r/1$ queue [44], validate the overall latency of Fig. 9. (a) and (b). And we find that

communication latency dominates the overall latency of a transaction during the validate phase, as shown in the decomposed latencies of Fig. 9. (a) and (b). Moreover, a larger batch size leads to a larger throughput of the committing peer in the validate phase, because a larger amount of data can be written to disk per IO. For example, the throughput of a committing peer with an HDD and a *BatchSize* of 20 is 85 transactions per second, and the throughput of a committing peer with an HDD and a *BatchSize* of 50 is much higher, i.e., 185 transactions per second. And an SSD storage device shows higher throughput than an HDD in the validate phase because of its smaller service time per block, as shown in the service time of the validate phase of Fig. 9. (a) and (b).

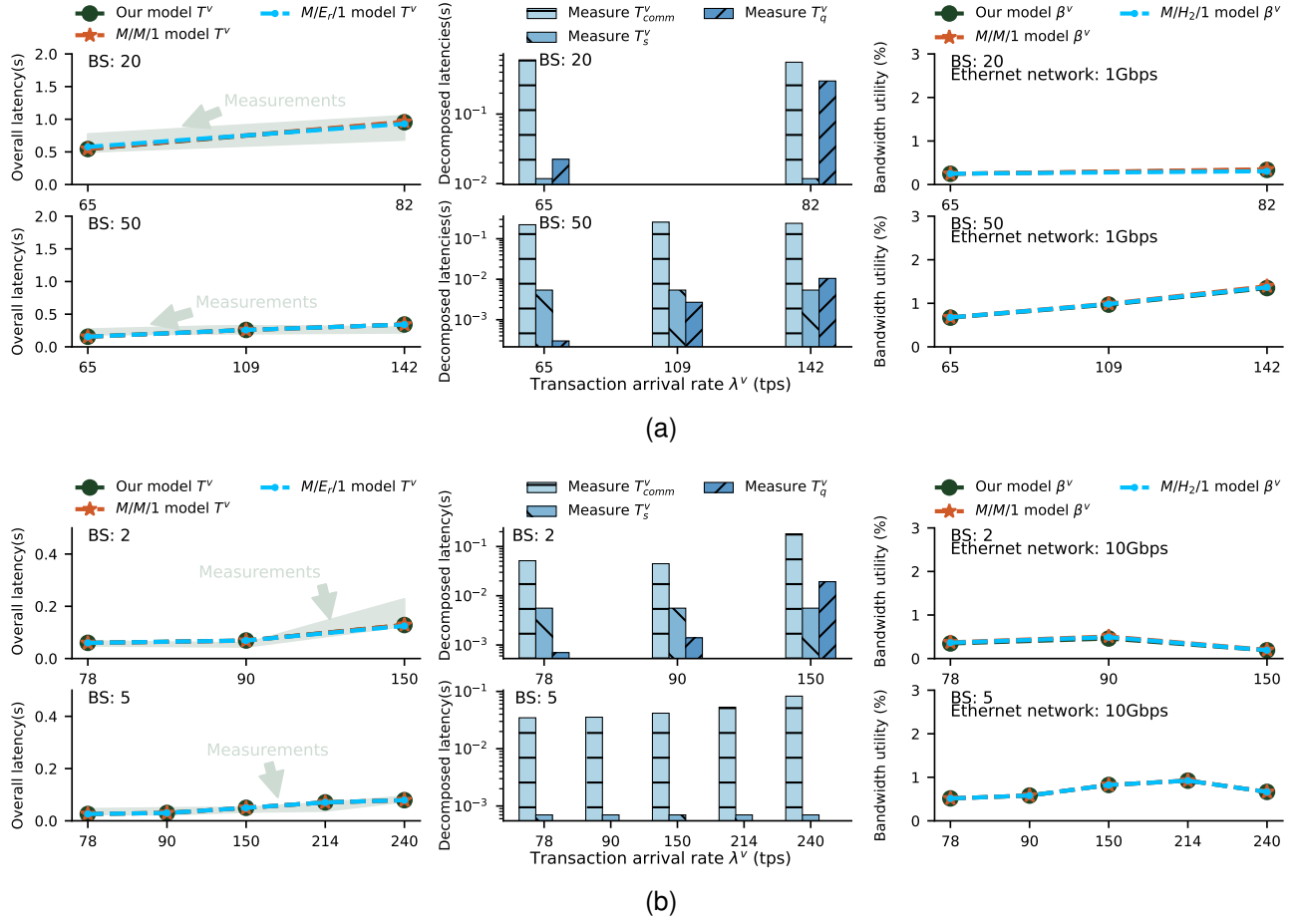


Fig. 9. The effects of batch size on latency (T^v), decomposed latencies (T_{comm}^v , T_s^v , T_q^v), and bandwidth utilization (β^v), where $T^v = T_{comm}^v + T_s^v + T_q^v$. There is a committing peer. The remaining machines are seven clients and three ordering service nodes. (a) In cluster one, the committing peer is with an HDD storage device. All machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 1 Gbps. (b) In cluster two, the committing peer is with an SSD storage device. All machines are connected via a bi-directional non-blocking switch to an Ethernet network with a speed of 10 Gbps.

E. Waiting-Time Reduction Strategy

The waiting-time reduction problem is modeled as an optimization problem in our proposed model, as shown in equation (4) and inequality (5), where a transaction must wait a certain time period called waiting time until a new block is created. The waiting-time reduction problem has many optimization parameters, including *BatchSize*, *BatchTimeout*, and transaction arrival rate. In practice, most existing works [24], [39] treat *BatchSize* as a key performance parameter. Thus, we focus on the *BatchSize* parameter, and propose a strategy to reduce the waiting time. Specifically, our waiting-time reduction strategy controls variates by adjusting the *BatchSize* parameter while fixing both the *BatchTimeout* and the transaction arrival rate parameters.

In the worst case, waiting time dominates a transaction latency in the order phase under unfit block-setting parameters. As shown in the decomposed latencies of Fig. 7. (a), waiting time dominates transaction latency in the order phase under unfit block-setting parameters, i.e., a transaction arrival rate of 109 transactions per second, a *BatchSize* of 50 transactions per block, and a *BatchTimeout* of 1 second per block. To reduce waiting time, we apply a waiting-time reduction strategy by

reducing the *BatchSize* from 50 to 20 while fixing the rest of the parameters. The experiment results of overall latency and decomposed latencies in Fig. 8. (a) show that waiting time does not dominate transaction latency in the order phase. For example, let the number of OSNs take 3, the waiting time is reduced from 0.2294 seconds in Fig. 7. (a) to 0.0917 seconds in Fig. 8. (a). The above experiment results demonstrate the validity of our waiting-time reduction strategy.

VI. CONCLUSION

In conclusion, this paper presents a client-driven measurement framework to characterize the performance of each decomposition phase in Hyperledger Fabric blockchain service. We thoroughly analyze and open-source the implementation details of the measurement framework. We have some interesting observations, e.g., 1) In the execute phase, an endorsing peer in the execute phase has scalable throughput with the number of CPU cores due to parallel service times among CPU cores in an endorsing peer. 2) In the order phase, a larger batch size leads to a more significant throughput of the OSN leader in the ordering service because it better utilizes network bandwidth, resulting in lower communication latency. 3) The Raft-based ordering service demonstrates good scalability with

the number of ordering service nodes, due to concurrent processing between the OSN leader and its followers. 4) The communication latency, particularly that of the Raft protocol, dominates the overall latency of a transaction spent in the order phase. 5) The waiting time would dominate a transaction's latency in an unfit block setting. Based on these observations, we propose a stochastic computation model for each phase's performance analysis and an alpha-beta communication model to analyze the corresponding communication latency. We then apply the model and propose a strategy to reduce waiting time when it dominates transaction latency in the order phase under unfit block-setting parameters. Finally, experiment results in two different clusters demonstrate the accuracy and validity of the proposed model.

REFERENCES

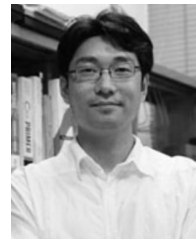
- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Available at <http://dx.doi.org/10.2139/ssrn.3440802>, 2008.
- [2] A. Sajjad, Q. Zhang, F. Ciampi, F. Pascucci, and B. Gupta, "What motivates bitcoin miners to practice bitcoin mining: An assessment based on behavioral reasoning theory," *IEEE Transactions on Engineering Management*, vol. 71, pp. 8209–8222, 2024.
- [3] N. Larionov and Y. Yanovich, "Shared send mixers untangling in bitcoin clustering heuristics adjustment," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2024, pp. 225–229.
- [4] Z. Liu, C. Hu, R. Zhao, P. Hu, A. Alrawais, and T. Xiang, "Blockchain assisted industrial data registration and reconstruction management scheme," *IEEE Transactions on Network Science and Engineering*, vol. 12, no. 3, pp. 2345–2359, 2025.
- [5] A. U. Rehman, N. Tariq, M. A. Jan, F. Khan, H. Song, and M. Ibrahim, "A blockchain-based hybrid model for iomt-enabled intelligent healthcare system," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 4, pp. 3512–3521, 2024.
- [6] S. Yuan, B. Cao, Y. Sun, Z. Wan, and M. Peng, "Secure and efficient federated learning through layering and sharding blockchain," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 3120–3134, 2024.
- [7] X. Yang, J. Hou, L. Xu, and L. Zhu, "zkfabledger: Enabling privacy preserving and regulatory compliance in hyperledger fabric," *IEEE Transactions on Network and Service Management*, vol. 22, no. 2, pp. 2243–2263, 2025.
- [8] V. S., P. M., N. S., and E. O. Martinson, "An efficient secure sharing of electronic health records using iot-based hyperledger blockchain," *International Journal of Intelligent Systems*, vol. 2024, no. 1, p. 6995202, 2024.
- [9] T. Sato, T. Shimosawa, and N. Yamai, "Concept of haopssc: Toward decentralized operations for ensuring high availability in consortium blockchain-based systems," in *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2025, pp. 1–3.
- [10] G. Spadavecchia, M. Fiore, M. Mongiello, and D. De Venuto, "Decentralizing iot data processing: The rise of blockchain-based solutions," in *2025 Design, Automation and Test in Europe Conference (DATE)*, 2025, pp. 1–2.
- [11] R. Karl, V. Mazzonello, B. Pacheco, J. Silveira, S. Schlarb, V. Siska, and A. Weissenfeld, "Building a data space for the public security domain using smart contracts," in *2024 4th Intelligent Cybersecurity Conference (ICSC)*, 2024, pp. 32–38.
- [12] C. Wang, X. Chu, and Y. Qin, "Dissecting mining pools of bitcoin network: Measurement, analysis and modeling," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 1, pp. 398–412, 2023.
- [13] J. Duan, L. Gu, W. Wang, and L. Wang, "Chainknot: Redactable blockchain protocol with forced synchronization," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 3091–3104, 2024.
- [14] B. A. M. González, O. Hasan, and L. Brunie, "Mitigation of sybil-based poisoning attacks in permissionless decentralized learning," in *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2025, pp. 1–5.
- [15] M. Xu, H. Guo, Y. Cheng, C. Liu, D. Yu, and X. Cheng, "Ec-chain: Cost-effective storage solution for permissionless blockchains," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
- [16] S. Moradi, K. E. Khorasani, and S. Rouhani, "Privacy-preserving and incentive-driven relay-based framework for cross-domain blockchain interoperability," in *2025 IEEE International Conference on Blockchain (Blockchain)*, 2025, pp. 207–216.
- [17] Y. Tang, J. Yan, C. Chakraborty, and Y. Sun, "Hedera: A permissionless and scalable hybrid blockchain consensus algorithm in multiaccess edge computing for iot," *IEEE Internet of Things Journal*, vol. 10, no. 24, pp. 21 187–21 202, 2023.
- [18] Z. Xu, S. Ahmad Salehi, and N. Chilamkurti, "Cryptography-based bootstrapping scheme for permissionless sharding blockchain," in *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, 2024, pp. 152–153.
- [19] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, "Escaping from consensus: Instantly redactable blockchain protocols in permissionless setting," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [20] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys '18. New York, NY, USA: Association for Computing Machinery, 2018.
- [21] S. Zhao, Z. Zhang, J. Wang, Y. Yuan, M. Zhang, G. Wang, and J. Xiao, "Efficient partial order based transaction processing for permissioned blockchains," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 1888–1901.
- [22] M. Zhang, W. Sun, H. Li, X. Wang, Z. Yang, C. Qu, and X. Li, "Manipulated transaction collision attack on execute-order-validate blockchain," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 2, pp. 1132–1149, 2025.
- [23] M. Yu, Y. Zhao, J. Wang, D. Hu, X. Liu, Q. Li, and K. Li, "Enabling high-performance eov blockchains via transaction ordering exploration," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*, 2024, pp. 356–366.
- [24] J. Enare Abang, H. Takruri, R. Al-Zaidi, and M. Al-Khalidi, "Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an iot perspective," *Internet of Things*, vol. 26, p. 101217, 2024.
- [25] H. Ding, X. Piao, Y. Hong, Y. Liu, H. Song, M. Liu, and S. Gao, "Dynamic optimization for trade-off in hyperledger fabric towards latency-sensitive iot services," in *2024 IEEE International Conference on Web Services (ICWS)*, 2024, pp. 899–909.
- [26] R. K. Kaushal and N. Kumar, "Exploring hyperledger caliper benchmarking tool to measure the performance of blockchain based solutions," in *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, 2024, pp. 1–6.
- [27] S. Abbas, A. Sultana, and W. Lin, "Enhancing throughput in hyperledger fabric through endorsement policy strategy," in *2024 IEEE/ACIS 27th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 2024, pp. 195–199.
- [28] A. Stoltidis, K. Choumas, and T. Korakis, "Performance optimization of high-conflict transactions within the hyperledger fabric blockchain," in *2024 6th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, 2024, pp. 1–4.
- [29] C. Melo, G. Gonçalves, F. A. Silva, and A. Soares, "Performance modeling and evaluation of hyperledger fabric: An analysis based on transaction flow and endorsement policies," in *2024 IEEE Symposium on Computers and Communications (ISCC)*, 2024, pp. 1–6.
- [30] Z. Yu, Z. Chang, L. Wang, and G. Min, "Contract-based incentive design for resource allocation in edge computing-based blockchain," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 6, pp. 6143–6156, 2024.
- [31] W. Fan, Z. Hao, B. Tang, F. Wu, and Y. Liu, "Resource matching for blockchain-assisted edge computing networks," *IEEE Internet of Things Journal*, vol. 11, no. 8, pp. 14 460–14 471, 2024.
- [32] Z. Yin, B. Bai, Y. Liu, and T. Cheng, "Research on distributed node resource optimization mechanism for multi-agent systems combined with blockchain technology," in *2024 IEEE International Conference on Blockchain (Blockchain)*, 2024, pp. 536–541.
- [33] Z. Wang, Q. Hu, R. Li, M. Xu, and Z. Xiong, "Incentive mechanism design for joint resource allocation in blockchain-based federated learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1536–1547, 2023.

- [34] Y. Zuo, W. Chen, J. Guo, W. Fan, B. Sheng, and S. Jin, "Fluid antenna-enabled integrated sensing, communication, and computing systems with mobile blockchain," *IEEE Transactions on Network Science and Engineering*, pp. 1–19, 2025.
- [35] R. K. Kausal, N. Kumar, S. Makka, and K. Saluja, "Demystifying hyperledger fabric framework for distributed ledgers and approach to evaluate its performance," in *2023 International Conference on Sustainable Emerging Innovations in Engineering and Technology (ICSEIET)*, 2023, pp. 116–120.
- [36] S. Lee, M. Kim, J. Lee, R.-H. Hsu, M.-S. Kim, and T. Q. S. Quek, "Facing to latency of hyperledger fabric for blockchain-enabled iot: Modeling and analysis," *IEEE Network*, vol. 37, no. 6, pp. 232–239, 2023.
- [37] K. S. S. Wai and N. N. M. Thein, "Performance analysis on block size valuation of hyperledger fabric blockchain," in *2023 IEEE Conference on Computer Applications (ICCA)*, 2023, pp. 50–55.
- [38] A. Augusto, R. Belchior, I. Kocsis, L. Gönczy, A. Vasconcelos, and M. Correia, "Cbdcc bridging between hyperledger fabric and permissioned evm-based blockchains," in *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2023, pp. 1–9.
- [39] H. Noh, S. Ji, Y. Go, G. Seok Park, and H. Song, "Resilient and fast block transmission system for scalable hyperledger fabric blockchain in multi-cloud environments," *IEEE Transactions on Network and Service Management*, vol. 21, no. 5, pp. 5118–5134, 2024.
- [40] X. Lu, Q. He, X. Wang, J. Lloret, P. Li, Y. Qian, and M. Huang, "A reputation-based energy-efficient transaction propagation mechanism for blockchain-enabled multi-access edge computing," *IEEE Transactions on Computers*, vol. 74, no. 11, pp. 3897–3910, 2025.
- [41] W. Canhui, "Hyperledger fabric phase decomposition framework," *Github*, 2025. [Online]. Available: <https://github.com/Canhui/FabricPhaseBench>.
- [42] P. Thakkar and S. Natarajan, "Scaling blockchains using pipelined execution and sparse peers," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 489–502.
- [43] P. Yuan, K. Zheng, X. Xiong, K. Zhang, and L. Lei, "Performance modeling and analysis of a hyperledger-based system using gspn," *Computer Communications*, vol. 153, pp. 117–124, 2020.
- [44] T. Meng, Y. Zhao, K. Wolter, and C.-Z. Xu, "On consortium blockchain consistency: A queueing network model approach," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 6, pp. 1369–1382, 2021.
- [45] U. Roy and N. Ghosh, "Fabman: A framework for ledger storage and size management for hyperledger fabric-based iot applications," *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 3140–3151, 2024.
- [46] C. Lu, J. Zhu, and L. Ouyang, "Mission reliability and optimization in multi-agent systems using hyperledger fabric platform," in *2024 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, 2024, pp. 1–6.
- [47] R. Alhabib and P. Yadav, "Hyperledger fabric platform for secure and efficient data sharing in autonomous vehicles," in *2024 11th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, 2024, pp. 1–9.
- [48] T. Nakaike, Q. Zhang, Y. Ueda, T. Inagaki, and M. Ohara, "Hyperledger fabric performance characterization and optimization using goleveldb benchmark," in *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2020, pp. 1–9.
- [49] C. Meese, H. Chen, W. Li, D. Lee, H. Guo, C.-C. Shen, and M. Nejad, "Adaptive traffic prediction at the its edge with online models and blockchain-based federated learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 9, pp. 10725–10740, 2024.
- [50] J. Wang, Y. Wang, X. Zhang, Z. Jin, C. Zhu, L. Li, R. Zhu, and S. Lv, "Learningchain: A highly scalable and applicable learning-based blockchain performance optimization framework," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1817–1831, 2024.
- [51] X. Xu, G. Sun, L. Luo, H. Cao, H. Yu, and A. V. Vasilakos, "Latency performance modeling and analysis for hyperledger fabric blockchain network," *Information Processing & Management*, vol. 58, no. 1, p. 102436, 2021.
- [52] M. Cavinkumaran, A. Srinivasan, and K. Vijayakumar, "Hyperledger iroha for a secure and efficient voting system," in *2024 2nd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, 2024, pp. 380–386.
- [53] L. Li, H. Wu, L. He, J. Yang, Z. Cao, and B. Düdler, "Matching as you want: A decentralized, flexible, and efficient realization for crowdsourcing with dual-side privacy," *IEEE Transactions on Network Science and Engineering*, vol. 12, no. 2, pp. 1026–1040, 2025.
- [54] L. He, H. Wu, L. Li, J. Yang, and L. Wu, "Unite⁺: Privacy-aware data quality assessment with public verifiability via federated learning in blockchain-empowered crowdsourcing," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 522–538, 2026.
- [55] C. Li, J. Meng, M. Dong, M. Huang, X. Xin, and K. Ota, "Cdpp: Cross-domain privacy-preserving for low-altitude networking with certificate-less dvps," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 4242–4260, 2026.
- [56] E. Zeydan, J. Baranda, J. Mangues-Bafalluy, S. S. Arslan, and Y. Turk, "A trustworthy framework for multi-cloud service management: Self-sovereign identity integration," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 3135–3147, 2024.
- [57] S. Abbas, A. Sultana, and G. Kaddoum, "Quantum-safe blockchain in hyperledger fabric," *IEEE Networking Letters*, vol. 7, no. 1, pp. 61–65, 2025.
- [58] S. Basu, D. Bera, and S. Karmakar, "Detection and intelligent control of cloud data location using hyperledger framework," *IEEE Transactions on Consumer Electronics*, vol. 69, no. 1, pp. 76–86, 2023.
- [59] S. Aslam, S. Aslam, T. Wang, D. Feng, and S. Zhang, "A novel and secure machine learning-based hyperledger blockchain for iot healthcare," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 31126–31140, 2025.
- [60] A. Alexandridis, G. Al-Sumaidae, Z. Zilic, G. Jeon, and J. Wang, "An iot ecosystem platform for the evaluation of blockchain feasibility," *IEEE Internet of Things Journal*, vol. 10, no. 24, pp. 21515–21527, 2023.
- [61] M. A. Abuhaliqa, C. Catal, and Q. Liu, "Securing the internet of things network using a hierarchical hyperledger fabric model," in *2023 IEEE International Conference on Metaverse Computing, Networking and Applications (MetaCom)*, 2023, pp. 775–779.
- [62] O. Wu, Z. Wang, and Z. Li, "Performance modeling of hyperledger fabric 2.0: A queueing theory-based approach," *Wireless Communications and Mobile Computing*, vol. 2023, no. 1, p. 9957995, 2023.
- [63] C. Melo, G. Gonçalves, F. A. Silva, I. Fé, E. Moura, A. Soares, E. Choi, D. Min, J.-W. Lee, and T. A. Nguyen, "Transactional dynamics in hyperledger fabric: A stochastic modeling and performance evaluation of permissioned blockchains," *ICT Express*, vol. 11, no. 1, pp. 87–92, 2025.
- [64] J. Enare Abang, R. Al-Zaidi, H. Takruri, and M. Al-Khalidi, "Intelligent latency optimization in hyperledger fabric for seamless metaverse integration," *Internet of Things*, vol. 35, p. 101835, 2026.
- [65] Y. Wang, Y. Li, W. Jiao, G. Wang, J. Zhao, Y. Qiang, and K. Li, "An efficient, secured, and infinitely scalable consensus mechanism for peer-to-peer energy trading blockchain," *IEEE Transactions on Industry Applications*, vol. 59, no. 5, pp. 5215–5229, 2023.
- [66] M. Cao, B. Cao, and M. Peng, "Decoupling intra- and inter-shard consensus for high scalability in permissioned blockchain," *IEEE Transactions on Networking*, vol. 34, pp. 1715–1729, 2026.
- [67] B. Li, T. Duan, Q. Zhao, Y. Guo, Z. Song, H. Zhang, Z. Li, and Y. Sun, "Performance modeling of relay chain," *IEEE Transactions on Networking*, vol. 33, no. 1, pp. 194–209, 2025.
- [68] A. K. Gupta, D. Raj, Y. K. Sharma, A. Sharma, M. K. Singh, and A. K. Agrawal, "Electronic healthcare data sharing application based on hyperledger consortium blockchain network," in *2025 International Conference on Networks and Cryptology (NETCRYPT)*, 2025, pp. 1405–1410.
- [69] J. Jia, O. Wu, S. Li, R. Ma, and H. Zhang, "A scheduling algorithm for hyperledger fabric based on transaction batch processing," in *2024 23rd International Symposium on Parallel and Distributed Computing (ISPD)*, 2024, pp. 1–8.
- [70] P. Martina, R. Dhanvardini, R. Vijay, R. Amirtharajan, and P. Pravinkumar, "Design development and execution of smart contract : An overview," in *2023 International Conference on Computer Communication and Informatics (ICCCI)*, 2023, pp. 1–5.
- [71] B. Li, T. Hu, X. Xu, and L. Wang, "Vulfinder: Exploring chaincode vulnerabilities more effectively and efficiently using knowledge graph based defect pattern matching," *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3247–3266, 2025.
- [72] S. Godbole and P. R. Krishna, "Poster: Verisol-mce: Verification-based condition coverage analysis of smart contracts using model checker engines," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024, pp. 434–437.
- [73] Y. Wang, J. Ali, J. Arshad, and Y. Liu, "A proxy-layer approach to secure smart contract deployment on private evm-based poa blockchains," in *2023 IEEE International Conference on Blockchain (Blockchain)*, 2023, pp. 109–112.

- [74] M. Zhu, L. Wang, and X. Li, "Design and optimization of smart contracts for cross-domain sharing of sensitive data," in *2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS)*, 2024, pp. 633–640.
- [75] B.-J. Ji and T.-W. Kuo, "Better clients, less conflicts: Hyperledger fabric conflict avoidance," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2024, pp. 368–376.
- [76] H. Huang, Z. Yin, Q. Yang, T. Li, X. Luo, L. Zhou, and Z. Zheng, "Scalability and security of blockchain-empowered metaverse: A survey," *IEEE Open Journal of the Computer Society*, vol. 5, pp. 648–659, 2024.
- [77] H. Huang, Z. Yin, Q. Chen, J. Zheng, X. Luo, G. Ye, X. Peng, Z. Zheng, and S. Guo, "Brokerchain: A blockchain sharding protocol by exploiting broker accounts," *IEEE Transactions on Networking*, vol. 33, no. 4, pp. 1930–1945, 2025.
- [78] W. Jie, W. Qiu, A. S. Voundi Koe, J. Li, Y. Wang, Y. Wu, J. Li, and Z. Zheng, "A secure and flexible blockchain-based offline payment protocol," *IEEE Transactions on Computers*, vol. 73, no. 2, pp. 408–421, 2024.
- [79] X. Lu, Z. Wu, and J. Cao, "Atmchain: Blockchain-based security architecture for air traffic management in future," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 60, no. 4, pp. 3872–3896, 2024.
- [80] B. Bellaj, A. Ouaddah, E. Bertin, N. Crespi, and A. Mezrioui, "Drawing the boundaries between blockchain and blockchain-like systems: A comprehensive survey on distributed ledger technologies," *Proceedings of the IEEE*, vol. 112, no. 3, pp. 247–299, 2024.
- [81] F. Zantalis, G. Koulouras, and S. Karabetos, "Blockchain technology: A framework for endless applications," *IEEE Consumer Electronics Magazine*, vol. 13, no. 2, pp. 61–71, 2024.
- [82] I. M. Ali and M. M. Abdallah, "On off-chaining smart contract runtime protection: A queuing model approach," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 8, pp. 1345–1359, 2024.
- [83] X. Ling, Y. Le, S. Chen, J. Wang, and X. Zhou, "Blockchain-enabled decentralized services and networks: Assessing roles and impacts," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 6, pp. 2141–2154, 2025.
- [84] P. Soltani and F. Ashtiani, "Analytical modeling and throughput computation of blockchain sharding," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 983–997, 2024.
- [85] Q. Ding, X. Yue, Q. Zhang, Z. Xiong, J. Chang, and H. Zheng, "Bc²fi: Double-layer blockchain-driven federated learning framework for agricultural iot," *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 4362–4374, 2025.
- [86] E. Zeydan, J. Mangues-Bafalluy, S. Arslan, Y. Turk, and K. Antevski, "Analysis and performance evaluation of blockchain consensus mechanisms for network sharing," *Distrib. Ledger Technol.*, Jul. 2025, just Accepted.
- [87] J. Xu, C. Wang, and X. Jia, "A survey of blockchain consensus protocols," *ACM Comput. Surv.*, vol. 55, no. 13s, Jul. 2023.
- [88] Y. Wang, L. Yuan, W. Jiao, Y. Qiang, J. Zhao, Q. Yang, and K. Li, "A fast and secured vehicle-to-vehicle energy trading based on blockchain consensus in the internet of electric vehicles," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 6, pp. 7827–7843, 2023.
- [89] G. S. Park and H. Song, "Watchdog: Network-aware consensus protocol for enhancing scalability of public blockchains," *IEEE Internet of Things Journal*, vol. 11, no. 11, pp. 20 138–20 151, 2024.
- [90] A. Sokhankhosh and S. Rouhani, "Proof-of-collaborative-learning: A multi-winner federated learning consensus algorithm," in *2024 IEEE International Conference on Blockchain (Blockchain)*, 2024, pp. 370–377.
- [91] C.-P. Chien, B.-H. Chia, C.-Y. Chang, S.-C. Lin, I. H.-R. Jiang, C. Liu, and C.-W. Lin, "Passing-order decision for three-to-two lane merging of connected and autonomous vehicles," in *2025 IEEE 31st International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2025, pp. 12–21.
- [92] X. Xue, D. Wei, and S. Huang, "A novel tpmbm filter for partly resolvable multitarget tracking," *IEEE Sensors Journal*, vol. 24, no. 10, pp. 16 629–16 646, 2024.
- [93] M. Di Mauro, G. Galatro, M. Longo, F. Postiglione, and M. Tambasco, "Performability analysis of containerized ims through queueing networks and stochastic models," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 2022, pp. 1–8.
- [94] C. Zhang, M. Yu, W. Wang, and F. Yan, "{Mark}: Exploiting cloud services for {Cost-Effective}, {SLO-Aware} machine learning inference serving," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 1049–1062.
- [95] L. De Simone, M. D. Mauro, R. Natella, and F. Postiglione, "A latency-driven availability assessment for multi-tenant service chains," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 815–829, 2023.
- [96] X. Zhong, A. M. Prakash, L. Petty, and R. A. James, "Bottleneck analysis to reduce primary care to specialty care referral delay," *IEEE Transactions on Automation Science and Engineering*, vol. 16, no. 1, pp. 61–73, 2019.



Canhui Wang received the B.E. degree in Software Engineering from Shandong University, P.R. China, in 2016, and the Ph.D. degree in Computer Science from Hong Kong Baptist University in 2023. He is currently a lecturer (research and teaching track) with the Department of Computer Science and Technology at Huaqiao University, Xiamen. He was a senior engineer with the Hong Kong Applied Science and Technology Research Institute, working on Applied Cryptosystems Trust and AI Technologies. His research interests include performance evaluation, blockchain, and security. He is a member of the IEEE.



Xiaowen Chu received the B.E. degree in Computer Science from Tsinghua University, P.R. China, in 1999, and the Ph.D. degree in Computer Science from the Hong Kong University of Science and Technology in 2003. He is currently a full professor with the Data Science and Analytics Thrust of The Hong Kong University of Science and Technology (Guangzhou). His research interests include parallel and distributed computing, cloud computing, and wireless networks. He is serving as an Associate Editor of IEEE Transactions on Network Science and Engineering and IEEE Transactions on Cloud Computing, and has served as an Associate Editor of IEEE Internet of Things Journal, and a Guest Editor of IEEE Network and IEEE Transactions on Industrial Informatics. He is a fellow of the IEEE.